

Earthquake Co-Occurrence Analysis with Apache Spark

Scalable and Cloud Programming — Project Report, A.Y. 2025/2026

Matteo Fontana — University of Bologna

Source code: <https://github.com/ma3ti/Earthquake-Co-Occurrence-Analysis-with-Apache-Spark>

1 Problem and dataset

The goal of the project is to find, in a large catalogue of earthquake events, the *pair of geographic locations whose earthquakes co-occur most frequently on the same day*, and to print that pair together with the list of its co-occurrence dates in ascending order. Following the assignment specification, an event’s location is discretised by rounding latitude and longitude to the first decimal digit (half-up rounding), so that each rounded pair identifies a *cell* of roughly 11×9 km at mid latitudes; the time of an event is discretised to its calendar day; and duplicate events — multiple earthquakes in the *same cell on the same day* — must be counted once, otherwise the most frequent “pair” would degenerate into a cell paired with itself.

The full dataset contains **3 445 751 events** (≈ 175 MB of CSV) with header `longitude,latitude,date`, covering the years 1990–2023. Columns are read by name, so their order is irrelevant; the day is extracted as the first ten characters of the timestamp. All Google Cloud experiments use the full dataset, as required.

2 Approach: two MapReduce implementations

Both implementations are pure RDD (MapReduce) pipelines that share the same preprocessing, accept the same arguments — the dataset URI and an optional number of partitions applied with `repartition` — and print byte-identical output, so their performance can be compared directly.

Common preprocessing. Each CSV row is mapped to a pair $(day, (lat, lon))$, with coordinates rounded via `BigDecimal(...).setScale(1, HALF_UP)` and the day taken as `date.substring(0,10)`.

2.1 V1 — self-join

V1 is the direct translation of the problem statement into MapReduce:

1. **Deduplicate:** `distinct()` keeps one event per (day, cell).
2. **Self-join on the day:** `parsed.join(parsed)` generates, for every day, *all ordered pairs* of cells active on that day.
3. **Filter:** only pairs with $loc_1 < loc_2$ (lexicographic on the string form) are kept, discarding self-pairs (A, A) and one of each symmetric pair $(A, B)/(B, A)$.
4. **Aggregate:** each surviving pair is mapped to $((loc_1, loc_2), List(day))$ and date lists are concatenated with `reduceByKey(_ ++ _)`.
5. **Select:** `sortBy(_._2.size, desc).first()` returns the winning pair; its dates are sorted and printed.

V1 is correct but has three structural inefficiencies. For a day with k active cells the join *materialises* k^2 pairs before the filter discards more than half of them; the aggregation drags entire date *lists* through the shuffle, with payload growing as counts accumulate; and a *global sort* (one extra shuffle, preceded by a sampling job for the range partitioner) is used merely to select a maximum. The pipeline performs four shuffles: `distinct`, `join`, `reduceByKey`, `sortBy`.

2.2 V2 — per-day combinations

V2 removes all three inefficiencies by never joining the dataset with itself:

1. **Group and deduplicate in one shuffle:** `aggregateByKey(Set)` builds, per day, the *set* of active cells — the set semantics subsume `distinct()`. The resulting RDD has one record per distinct day (order of 10^4 records, more than two orders of magnitude smaller than the input) and is cached with `persist(MEMORY_AND_DISK)` for reuse.
2. **Generate each pair exactly once:** per day, the cell set is sorted with the same lexicographic key used by V1 and streamed through `combinations(2)`, a *lazy iterator* that yields each of the $\binom{k}{2}$ pairs once, already ordered: no symmetric pairs, no self-pairs, no post-filter, and no materialisation of the pair set in memory.
3. **Count, don’t collect:** pairs are counted with `reduceByKey(_ + _)` on integers, benefiting from map-side combining — the shuffle carries one `Int` per distinct pair per map task instead of growing date lists.
4. **Maximum via reduce:** per-partition maxima are combined in a single action; no global sort, no sampling job.

5. **Recover the winner’s dates:** the *cached* per-day RDD is filtered for days whose set contains both winning cells ($O(1)$ set lookups), and the $\approx 10\text{k}$ resulting date strings are collected and sorted on the driver.

V2 performs two shuffles (`aggregateByKey`, `reduceByKey`). Table 1 summarises the differences.

	V1 (self-join)	V2 (combinations)
Shuffles	4 (<code>distinct</code> , <code>join</code> , <code>reduceByKey</code> , <code>sortBy</code>)	2 (<code>aggregateByKey</code> , <code>reduceByKey</code>)
Pairs generated per day (k cells)	k^2 , filtered to $\binom{k}{2}$ afterwards	$\binom{k}{2}$, streamed lazily
Shuffle payload of the aggregation	concatenated date Lists	Int counts, map-side combined
Finding the maximum	global sort + sampling job, then <code>first()</code>	single <code>reduce</code>
Dates of the winning pair	carried through the whole pipeline	filter on the cached per-day RDD

Table 1: Structural comparison of the two implementations.

3 Experimental setup

All experiments ran on Google Cloud Dataproc (region `eu-west-1`) with one master and 2, 3 or 4 workers, all `n2-standard-4` machines (4 vCPUs, 16 GB RAM) with explicit 240 GB boot disks, as mandated by the assignment. The application is packaged as a single fat JAR (sbt-assembly, Scala 2.12, Spark dependencies *provided* by the Dataproc runtime) containing both classes; dataset and JAR reside in a Cloud Storage bucket.

Each cluster size was tested with Spark’s *default* partitioning and with explicit `repartition` values chosen as multiples of the total worker cores ($1\times$, $2\times$, $4\times$, up to $8\times$ on the largest cluster). The full matrix — 15 V1 and 9 V2 configurations — was executed by two scripts committed in the repository, which for every cluster size *create the cluster* $\rightarrow$ *submit the jobs* $\rightarrow$ *delete the cluster*, guaranteeing reproducibility and that no cluster is left running. The reported metric is the job wall-clock time from the Dataproc console (a screenshot of the complete job list is committed as evidence, together with the raw driver output of one run); it includes YARN application start-up (≈ 1 minute), which is identical across configurations. Because of the education-credit budget each configuration was run once; run-to-run variance is therefore not measured, and we flag below the one observation we attribute to it.

4 Result of the analysis

Every completed run — *both* algorithms, every cluster size and partitioning — produced the identical result:

((38.8, -122.7), (38.8, -122.8)) with **10 014 dates** from 1990-01-05 to 2023-07-29.

The date list was verified to be strictly ascending and duplicate-free. The agreement of two algorithmically independent implementations on all 21 successful runs is strong evidence of correctness.

The result is also geophysically meaningful. The two adjacent cells cover **The Geysers** geothermal field in Northern California, the largest geothermal complex in the world and the most seismically active area of the United States: fluid injection induces microearthquakes essentially every day. Indeed the pair co-occurs on 82 % of all days in the 33-year window, with density *increasing* over time (62 % of days in the 1990s, 88 % in the 2000s, 91 % in the 2010s, 93 % in 2020–23), consistent with the documented intensification of injection operations — an independent sanity check that the pipeline measures a real phenomenon rather than an artefact.

5 Performance and scalability analysis

Table 2 reports the complete experimental matrix; Figures 1 and 2 visualise it.

5.1 Default partitioning: the cluster does not matter if the job cannot use it

The 175 MB CSV is split by Spark into $\lceil 175/128 \rceil = 2$ input partitions (128 MB being the default split size). This throttles the whole pipeline. In V2 the map side of the first shuffle — parsing 3.4 M rows and pre-aggregating — runs as *two tasks* regardless of the cluster, and dominates the runtime: the default-partitioning runs take 27 m 03 s, 26 m 08 s and 26 m 34 s on 2, 3 and 4 workers respectively — *flat*. Paying for twice the hardware buys nothing, because at most 2 of the 8–16 cores ever work.

V1 fares even worse: the RDD `join` inherits the 2-partition hash partitioner produced upstream by `distinct()` (Spark discards an existing partitioner only when the default parallelism exceeds it by more than an order of magnitude), so even the quadratic join and the aggregation run as *two mammoth tasks*. These tasks are at the edge of memory and survive only by heavy spilling, which makes them slow *and*

Workers	Cores	Partitions	V1 (self-join)	V2 (combinations)	V2 speedup
4	16	default	1 h 16 m	26 m 34 s	2.9×
		8	39 m 40 s	—	
		16	24 m 33 s	6 m 46 s	3.6×
		32	14 m 45 s	6 m 45 s	2.2×
		64	10 m 07 s	—	
		128	8 m 57 s	—	
3	12	default	49 m 26 s	26 m 08 s	1.9×
		12	failed (OOM)	8 m 04 s	completes
		24	19 m 16 s	8 m 01 s	2.4×
		48	13 m 45 s	—	
2	8	default	37 m 26 s	27 m 03 s	1.4×
		8	failed (OOM)	28 m 39 s	completes
		16	failed (OOM)	16 m 17 s	completes
		32	25 m 29 s	—	
		64	19 m 28 s	—	

Table 2: Wall-clock job time for every configuration (full dataset, n2-standard-4 workers).



Figure 1: Execution time vs. number of partitions (log-log), one panel per cluster. Default partitioning is plotted at $x \approx 2$, its effective input parallelism. Red crosses mark V1 runs aborted by executor OOM, placed at the time of abort.

unstable: 37, 49 and 76 minutes on 2, 3 and 4 workers. We attribute this inversion (more workers, slower) to run-to-run variance of a memory-stressed computation rather than to a systematic cause — with a single run per configuration the three values should be read as “ ≈ 40 –75 min, dominated by spill/GC behaviour”, against V2’s stable ≈ 26.5 min.

Explicit repartitioning removes the ceiling and is by far the largest single factor observed: on 4 workers it improves V1 by $8.5\times$ (1 h 16 m $\rightarrow$ 8 m 57 s) and V2 by $3.9\times$ (26 m 34 s $\rightarrow$ 6 m 45 s).

5.2 The V1 failures are algorithmic, not a cluster limit

Three V1 configurations — 3w-12p, 2w-8p, 2w-16p — failed with executors repeatedly killed by YARN (container exit code 137 = SIGKILL for exceeding the memory allocation) until a task exhausted its four attempts and the job aborted. The failures follow a precise pattern: they occur when partitions are *coarse* (1–2 per core), so that each join task materialises the k^2 pair explosion and the growing date lists for a large slice of days, *and* the cluster is small, so that several such tasks run concurrently on each executor and share its memory pool. Finer partitions shrink the per-task working set (2w-32p and 2w-64p complete), and on the 4-worker cluster even 8 coarse partitions survive because only ~ 2 tasks share each executor.

The decisive evidence that the limit is algorithmic comes from V2: *all three failing configurations complete under V2 on identical clusters* (8 m 04 s, 28 m 39 s, 16 m 17 s). Streaming pair generation and integer counters keep the per-task working set small no matter how coarse the partitioning. For a practitioner this is the more important property: V2 is not only faster, it is *robust* across the configuration space, while V1 requires tuning merely to terminate.

5.3 Partition sensitivity: memory-bound vs. compute-bound

Figure 1 shows two different response curves. **V1 keeps improving up to 8 partitions per core** (4 w: 39.7 → 24.6 → 14.8 → 10.1 → 9.0 minutes for 8 → 128 partitions): extra partitions primarily *relieve memory pressure* — smaller join state, less spill — so returns diminish slowly. **V2 plateaus at ≈ 1 partition per core**: on 4 workers, 16 and 32 partitions are indistinguishable (6 m 46 s vs. 6 m 45 s), and on 3 workers 12 and 24 partitions likewise (≈ 8 m). Once every core has work, V2 is compute-bound in the pair-generation stage and additional partitions only add scheduling overhead. The one V2 outlier is 2w-8p (28 m 39 s, no better than default): with 1 coarse partition per core on the smallest memory budget, the map-side combiners of the pair-count stage spill and the unlucky tasks that receive several seismically “busy” days become stragglers with no idle cores to absorb them; 16 partitions fix both effects (16 m 17 s).

A striking consequence of the plateau: **V2 with 16 partitions (6 m 46 s) beats V1’s best configuration with 128 partitions (8 m 57 s)** — a better algorithm outperforms brute-force tuning of a worse one, while shuffling a fraction of the data.

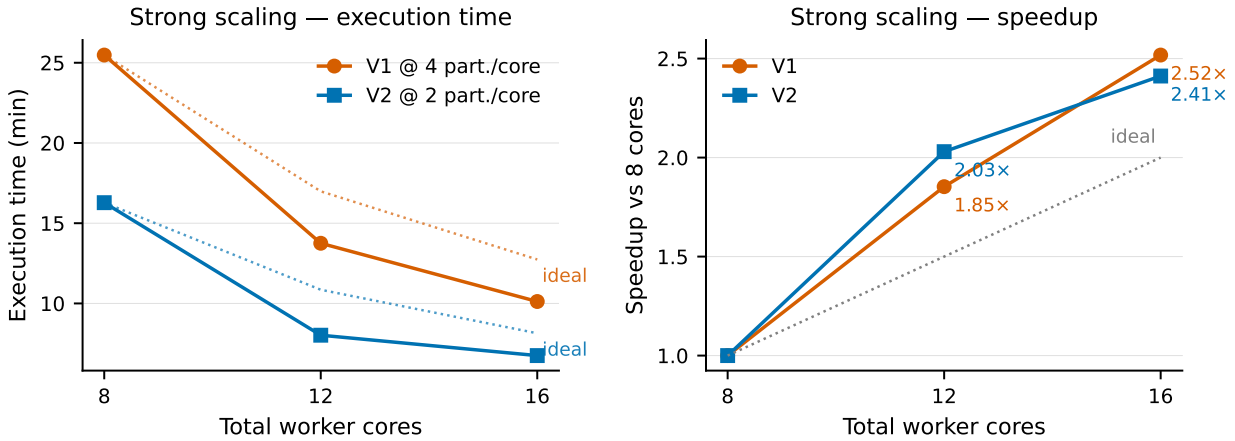


Figure 2: Strong scaling at fixed partitions-per-core (V1 at 4/core: 32/48/64 partitions; V2 at 2/core: 16/24/32 partitions). Dotted lines show ideal scaling from the 8-core baseline.

5.4 Strong scaling

Figure 2 isolates cluster-size scaling by holding partitions-per-core constant. Both versions scale well, and *superlinearly* on the first step: going from 8 to 12 cores (1.5 $\times$ resources) yields 1.85 $\times$ (V1) and 2.03 $\times$ (V2) speedups. Superlinearity is expected here and has a standard explanation: aggregate cluster memory grows with the worker count, so shuffle and aggregation state that spilled to disk on 2 workers stays in memory on 3, removing I/O that the ideal-scaling model does not account for. On the second step (12→16 cores) scaling becomes sublinear — 2.52 $\times$ (V1) and 2.41 $\times$ (V2) cumulative vs. the ideal 2.0 $\times$ from 8 cores, i.e. the marginal gain of the fourth worker is small for V2 (8.0 → 6.8 min, 1.19 $\times$ for 1.33 $\times$ cores). Two floors explain this: the fixed ≈ 1 minute of YARN/Spark start-up, and *data skew* — The Geysers region makes some days far heavier than others, and the pair generation for the busiest days executes inside single tasks that no amount of extra workers can subdivide. For this workload, 3 workers with correct partitioning (8 m) are the resource-efficiency sweet spot; the fourth worker buys 1.3 minutes.

6 Conclusions

Both implementations solve the assignment on the full dataset and agree, across 21 successful runs, on a geophysically validated result (The Geysers cell pair, 10014 dates). The performance study supports three conclusions. First, **partitioning dominates**: for a small file the default 2-split parallelism idles the cluster, and no hardware can compensate for it (V2 default is flat at ≈ 26.5 min on every cluster). Second, **the algorithm matters more than the hardware**: replacing the self-join with per-day streamed combinations cuts shuffles from 4 to 2, gives 1.4–3.6 $\times$ speedups at equal configuration, lets a mid-size configuration beat the best brute-force tuning of V1, and — most importantly — turns three OOM-failing configurations into successes on identical clusters. Third, **scaling is real but bounded**: near-ideal (even superlinear, via reduced spill) up to 12 cores, then limited by start-up overhead and by the intrinsic skew of the data. The best observed configuration is V2 on 4 workers with 32 partitions: **6 m 45 s**, an 11.3 $\times$ improvement over the naive algorithm with default partitioning on the same cluster.